

先直接按你要的两个部分来：

- 1) 根据这些实验讲义，把 **Verilog** 里跟考试最相关的语法点系统捋一遍。
- 2) 给你几段带错误的小代码，让你自己找问题（我先不给答案，我们可以之后一起对）。

一、根据实验讲义整理的 Verilog 重要语法点

我会按「考试容易出题 / 容易犯错」的角度来列。

1. 标识符（名字）和数字写法

- 大小写敏感：`sum` 和 `Sum` 是两个不同的名字。
 - 名字开头不能是数字：
 -  `adder2`, `_temp$1`
 -  `123port`（讲义里也专门说过这种是错的）
 - 只能用：`a-z` `A-Z` `0-9` `_` `$`，不能有空格、汉字、`-` 等。
 - 数字常量的格式：
位宽'进制(value)
 - 进制：`b/B` 二进制，`o/O` 八进制，`d/D` 十进制，`h/H` 十六进制
 - 例子：
 - `4'b0101`
 - `8'hA5`
 - `6'd15`
 - 四值逻辑：`0`, `1`, `x`, `z`
 - `x/X`：未知
 - `z/Z`：高阻
 - 注意：`0` 和 `1'b0` 在严格语义上不是完全一样（位宽不同），讲义里也提到过这一点。
-

2. module 与端口声明

基本骨架：

```
1 module my_circuit (  
2     input  [3:0] a,  
3     output [3:0] y  
4 );  
5     // ...  
6 endmodule
```

两种写法你都见过：

1. ANSI 风格（推荐）——在括号里直接写方向和位宽：

```
1 module sub_wr(  
2     input  in1, in2,  
3     output out1, out2  
4 );  
5 endmodule
```

1. 旧风格——括号里只写名字，后面再声明方向和位宽：

```
1 module sub_wr( in1, in2, out1, out2 );  
2     input  in1, in2;  
3     output out1, out2;  
4 endmodule
```

常见考点 / 易错点：

- 模块有端口，就必须在括号里写端口列表，不能留空：

```
1 module sub_wr(); // ❌ 错：有端口却不给在括号里列  
2     input in1, in2;  
3     output out1, out2;  
4 endmodule
```

- 括号里的端口名 必须和下面声明一致，不能漏写、不能多写。
 - 端口顺序：
 - 位置连接（`u1(a,b,c,y)`）要非常小心顺序；
 - 命名连接（`.a(a)`）更安全，考试也可能问。
-

3. wire vs reg（大重点）

讲义反复强调的点：

- **wire** —— 连线，本质是「被别的东西驱动」的网：
 - 典型驱动方式：
 - **assign** 连续赋值
 - 其他模块 / 原语门的输出
 - 不能作为 **initial** / **always** 左值。
- **reg** —— 寄存器/变量，用来在过程块（**initial** / **always**）里保存值：
 - 作为 **always** / **initial** 里的 左边：

```
1 | reg q;  
2 | always @(*) q = a & b;
```

- 也可以绑定到 **output** 端口：

```
1 | output reg [1:0] F;
```

端口和类型的组合（按讲义要求）：

- **input** :
 - 声明上应该是 **wire** 类型（不能在本模块里给它赋值）
- **output** :
 - 如果用 **assign** 驱动 → 默认 **wire**

- 如果在 `always` / `initial` 中赋值 → 写成 `output reg`，对应的内部变量是 `reg`

错误示例（讲义里有类似题目）：

```
1 module bad(  
2     input reg a,    // ✗ input 不应声明成 reg  
3     output y  
4 );  
5     always @(*) a = ~a; // ✗ 给 input 赋值  
6 endmodule
```

4. 连续赋值 `assign` 与 过程赋值 `always` / `initial`

4.1 连续赋值（组合逻辑）

```
1 wire y;  
2 assign y = a & b;
```

- 只能在模块顶层使用（不能写在 `always` 里面）。
- 左边必须是 `wire`（或其他 `net`）。
- 用来描述组合逻辑，和电路是「一直驱动」的感觉对应。

4.2 过程块 + 过程赋值

```
1 reg y;  
2 always @(*) begin  
3     y = a & b;  
4 end
```

- 过程块：`initial` / `always`
- 里面的赋值叫过程赋值，左边必须是 `reg`。

- 两种赋值符号：
 - 阻塞赋值 `=` —— 顺序执行，用在组合逻辑里比较常见。
 - 非阻塞赋值 `<=` —— 事件触发时并行更新，用在时序逻辑（触发器）里。

讲义给的建议：

- 组合逻辑： `assign` 或 `always @(*)` + 阻塞赋值 `=`
- 时序逻辑： `always @(posedge clk)` / `@(negedge clk)` + 非阻塞赋值 `<=`
- 不要在同一个 `always` 里混用 `=` 和 `<=`。

5. `initial` vs `always`，敏感列表

- `initial`：
 - 只在仿真里跑一次，从时间 0 开始。
 - 平时用在 `testbench` 里写激励、`$display`、`$finish` 等。
 - 设计模块里一般不允许用 `initial`（考试很可能拿这个来出错题）。
- `always`：
 - 有触发条件的时候会一遍又一遍执行。
 - 组合逻辑： `always @(*)` 或 `always @(a or b or c)`
 - 时序逻辑： `always @(posedge clk)` 或 `always @(negedge clk)`，也可以加 `rst`：

```
1  always @(posedge clk, posedge rst) begin
2      if (rst) q <= 1'b0;
3      else    q <= d;
4  end
```

敏感列表的规范（讲义 Lab8 有表格）：

- ☒ `always @*` / `always @(*)` —— 组合逻辑
- ☒ `always @(posedge clk)` / `@(negedge clk)` —— 时序逻辑
- ☒ `always @(posedge clk, posedge rst)` / `@(posedge clk, negedge rst)` ——

带异步复位的触发器

- **✗** `always @(posedge clk, a, b)` —— 不要把时钟和普通信号混在同一个 `always` 里。
 - **✗** `always @(posedge clk, negedge clk)` —— 同一时钟的两个边，都在一个块里，不合理。
-

6. 运算符：位运算 vs 逻辑运算（Lab3 内容）

- 位运算：`~ & | ^ ~^ ^~` —— 按位作用，输入是多位向量时，每一位各算各的。
- 逻辑运算：`! && ||` —— 把整个向量当成一个布尔值：
 - $0 \Rightarrow \text{false}$ ，非 $0 \Rightarrow \text{true}$
 - 返回值是 1 bit（0 或 1）。

优先级（从高到低）（只要记住大致顺序就行）：

1. `~`、`!`
2. `&`
3. `^`、`~^`、`^~`
4. `|`
5. `&&`
6. `||`

所以像：

```
1 | assign f = a & b & c | ~a & b;
```

实际上等价于：

```
1 | assign f = (a & b & c) | (~a & b);
```

7. 条件语句与 `case / casex / casez` (Lab4、Lab5)

- `if / else` :
 - 只能出现在 `initial` / `always` 中。
 - 左右可以用阻塞或非阻塞赋值（按上一节规则选择）。
- 条件运算符 `?:` :
 - 既可以写在 `assign` 里，也可以写在 `always` 里：

```
1 | assign y = sel ? a : b;
```

- `case` : 精确匹配
 - 最好配一个 `default`，否则有可能综合出锁存器（slides 里强调过）。
- `casex` : `x` 和 `z` 都被当成「无关位」匹配
- `casez` : 只把 `z` 当无关位，`x` 仍然是未知

重点注意：

- 没有 `default`，且并非所有输入都在 `case` 中列出来时：
 - 若是组合逻辑，综合器会认为你在「保持原值」，从而生成锁存器（讲义 lab7 latch 的例子）。
- `casex` 容易掩盖掉 `x`，仿真调试时可能看不到未知状态，这一点实验讲义里专门强调过。

8. 结构化设计：原语门 & 子模块实例化

- 原语门 (primitive) :

```
1 | and u1 (w1, a, b); // w1 = a & b
2 | not u2 (na, a);    // na = ~a
3 | or  u3 (y, w1, c); // y = w1 | c
```

语法: `门类型 实例名 (输出, 输入1, 输入2, ...)`

- 子模块实例化：

```
1 | my_adder u1(.a(a), .b(b), .sum(sum));
```

常见错误点：

- 实例名不能重复：同一层级里不能有两个 `u1`。
- 实例化不能写在 `always`、`initial` 里面（不能把结构化建模嵌进行为块）。
- 位置连接时，端口顺序写错会导致功能错误；命名连接能避免。

9. Testbench 专用语法（考试也经常问）

- `timescale`：

```
1 | `timescale 1ns / 1ps
```

- 时间延迟：`#10`（在当前 `timescale` 下代表 10 单位）
- `initial` + 延迟/循环 产生激励：

```
1 | initial begin
2 |     {a,b,c} = 3'b000;
3 |     repeat(7) #10 {a,b,c} = {a,b,c} + 1;
4 |     #10 $finish;
5 | end
```

- 循环：`forever`，`for`，`while`，`repeat`
- 系统任务：`$display`、`$monitor`、`$time`、`$finish` 等（只在仿真中有意义）。

10. 其他小语法点（也可能考）

- 连接运算 `{}`：


```
1 | assign y = {1'b0, a, 1'b0, b, 1'b0, c};
```

- 数组 / 存储器:

```
1 | reg [2:0] mema [4:0];    // 5 个 3bit 单元
2 | mema[1] = 3'b101;
```

- 注释:

- 单行: `// ...`

- 多行: `/* ... */`

二、练习：找 Verilog 代码中的错误

你刚好在准备期中考试，我给你几段 **非常贴近讲义风格** 的小代码，每一段里都有 2–4 个错误（有的是语法错，有的是「设计规则」错，比如 `input` 写 `reg`、或者不该在设计模块里用 `initial` 等）。

 建议：先自己认真找一遍，然后你可以把你认为的错误列出来，我再和你一起对。

练习 1: `module` 与端口声明、数字常量

目标：实现 `y = a & b`。

```
1 | module 2and(    // Q1
2 |     input  a,b,
3 |     output y
4 | )
5 |     wire y;      // Q2
6 |     assign y = a & b    // Q3
7 | endmodule
```

你可以先数一数：这里一共有几类问题？每一行哪里不太对？

练习 2: wire / reg 与 always / assign 的使用

目标: 组合逻辑 `y = a & b | ~a & b`。

```
1 module bad_comb(  
2     input  a,  
3     input  b,  
4     output reg y  
5 );  
6     wire y;                // Q1  
7     always @(*) begin  
8         assign y = a & b;  // Q2  
9         y <= ~a & b;       // Q3  
10    end  
11 endmodule
```

想想:

- 哪些地方违反了「assign 只能给 wire」「always 里左边必须是 reg」「非阻塞赋值的使用场景」这些规则?

练习 3: if / case 与 latch 问题

目标: 2-1 选择器: `y = sel ? a : b`。

```

1  module mux2(
2      input      a,
3      input      b,
4      input      sel,
5      output reg y
6  );
7      always @(sel) begin          // Q1
8          if (sel == 1)            // Q2
9              y = a;
10         // else 分支漏了          // Q3
11     end
12 endmodule

```

思考:

- 这个 `always` 的敏感列表合不合理?
- `if (sel == 1)` 写法有没有隐患?
- 如果 `sel == 0` 的时候, 这里会发生什么? 综合器会给你什么硬件?

练习 4: 模块实例化与 `testbench` 的边界

有一个已经写好的模块:

```

1  module adder2(
2      input  [1:0] a, b,
3      output [2:0] sum
4  );
5      // ... 省略实现
6  endmodule

```

下面是某同学写的 `top` 模块, 用来在按下按钮时把 `a+b` 输出到 LED 上:

```

1  module top(
2      input      clk,
3      input  [1:0] sw_a,

```

```
4     input  [1:0] sw_b,  
5     output [2:0] led  
6 );  
7     reg  [1:0] a, b;  
8     reg  [2:0] sum;  
9  
10    always @(posedge clk) begin  
11        adder2 u1(.a(sw_a), .b(sw_b), .sum(sum)); // Q1  
12        led = sum;                                // Q2  
13    end  
14 endmodule
```

试着找一找：

- `adder2` 应该放在什么位置？
- `led` 应该是什么类型？现在这种写法有什么问题？

如果你愿意，我们可以这样继续练：

1. 你先从 **练习 1** 开始，把你认为的错误逐条写给我（比如：Q1: xxx 不对，因为……）。
2. 我帮你一条条对，并顺便讲讲这个错误对应的考点。
3. 然后我们再继续做练习 2、3、4。

Solution

我先直接把每一题的「错误点」和「参考正确写法」都给你，方便你对照练习。😊

练习 1 答案： module 与端口声明、数字常量

原代码：

```
1 module 2and( // Q1
2     input a,b,
3     output y
4 )
5     wire y; // Q2
6     assign y = a & b // Q3
7 endmodule
```

存在的问题

1. Q1: 模块名非法 + 少了分号

```
1 module 2and(
2     ...
3 )
```

- 模块名不能以数字开头，`2and` 是非法标识符，应改成 `and2` 或其他合法名字。
- 模块头结尾缺少分号，正确写法应为：

```
1 module and2(
2     input a,
3     input b,
4     output y
5 );
```

2. Q2: 重复声明同名信号

```
1 output y
2 ...
3 wire y;
```

- 在 ANSI 风格端口列表里写了 `output y`，默认就是一个 `wire`。
- 再写 `wire y;` 就变成重复声明，综合器会报错或警告。
- 应删掉内部的 `wire y;`，或者把端口直接写成 `output wire y`，不要再额外声明。

3. Q3: 赋值语句缺少分号

```
1 | assign y = a & b // 少了 ;
```

- 每条语句末尾必须有 `;`，这里漏了一个。

参考正确写法

版本 1: 用连续赋值（最推荐）

```
1 | module and2(  
2 |     input  a,  
3 |     input  b,  
4 |     output y  
5 | );  
6 |     assign y = a & b;  
7 | endmodule
```

版本 2: 如果一定要写出类型

```
1 | module and2(  
2 |     input      a,  
3 |     input      b,  
4 |     output wire y  
5 | );  
6 |     assign y = a & b;  
7 | endmodule
```

练习 2 答案: wire / reg 与 always / assign

原代码:

```
1  module bad_comb(  
2      input  a,  
3      input  b,  
4      output reg y  
5  );  
6      wire y;                // Q1  
7      always @(*) begin  
8          assign y = a & b;  // Q2  
9          y <= ~a & b;      // Q3  
10     end  
11 endmodule
```

目标: 组合逻辑 $y = a \& b \mid \sim a \& b$ 。

存在的问题

1. Q1: 同名信号既是 reg 又是 wire

```
1  output reg y;  
2  wire y;
```

- 端口声明成 `output reg y`，又在模块内部声明 `wire y`，同名不同类型，是错误的。
- 应删掉内部的 `wire y`，只保留一种声明方式。

2. Q2: 在 always 里使用 assign，并且 assign 目标是 reg

```
1  always @(*) begin  
2      assign y = a & b;  
3      ...  
4  end
```

- `assign` 是连续赋值语句，只能在模块层级使用，不能放在 `always` 里面。
- 同时，`assign` 左边必须是 `wire` 类型，而这里的 `y` 是 `reg` 类型，也不符合「`assign` 只能给 `wire`」的规则。

3. Q3: 非阻塞赋值用在组合逻辑 + 功能也写错了

```
1 | y <= ~a & b;
```

- 在纯组合逻辑 `always @(*)` 中，通常使用阻塞赋值 `=`，而不是非阻塞 `<=`。非阻塞赋值更适用于时序逻辑 `always @(posedge clk)`。
- 更重要的是：
目标是 `y = a & b | ~a & b`，而原代码等价于：
 - 先用 `assign` 让 `y = a & b`（虽然写法本身就违法）
 - 再在 `always` 里把 `y` 设成 `~a & b`
最终 `y` 实际上只有 `~a & b`，并没有实现 **OR** 的功能。

参考正确写法

写成纯组合逻辑（推荐）

```
1 | module good_comb(  
2 |     input  a,  
3 |     input  b,  
4 |     output y  
5 | );  
6 |     assign y = (a & b) | (~a & b);  
7 | endmodule
```

或者坚持用 `always` 也可以：


```

1  module good_comb(
2      input  a,
3      input  b,
4      output reg y
5  );
6      always @(*) begin
7          y = (a & b) | (~a & b); // 组合逻辑用阻塞赋值 =
8      end
9  endmodule

```

这里的考点：

- **assign** 只能写在模块级，驱动 **wire**。
- **always** 块内只能对 **reg/integer** 等做过程赋值。
- 组合逻辑 **always @(*)** 推荐用阻塞赋值 **=**。

练习 3 答案：if / case 与 latch 问题

原代码：

```

1  module mux2(
2      input  a,
3      input  b,
4      input  sel,
5      output reg y
6  );
7      always @(sel) begin // Q1
8          if (sel == 1) // Q2
9              y = a;
10         // else 分支漏了 // Q3
11     end
12 endmodule

```

目标：2-1 选择器 `y = sel ? a : b`。

存在的问题

1. Q1: 敏感列表不完整，组合逻辑里会出问题

```
1 | always @(sel) begin
```

- y 实际上依赖于 a、b、sel 三个信号，但敏感列表只有 sel。
- 当 a 或 b 变化、sel 不变时，这个 always 块不会重新执行，仿真行为不再是纯组合逻辑，会表现成某种「记忆」。
- 在综合里，这类写法非常容易让工具推导出锁存器 (latch)。
- 正确写法应使用：

```
1 | always @(*) begin
2 |     ...
3 | end
```

或者完整列出 a、b、sel。

2. Q2: if (sel == 1) 数字常量写法不够严谨

```
1 | if (sel == 1)
```

- 对于 1-bit 的选择信号，更好的写法是：
 - if (sel)，或者
 - if (sel == 1'b1)
- 1 是无大小、32 位的整型常量，会做宽度扩展再比较。对 1-bit 信号来说结果通常没问题，但从风格和考试角度，会希望你写出带宽度的常量。

3. Q3: 缺少 else 分支，导致 latch

```
1 | if (sel == 1)
2 |     y = a;
3 | // 没有 else 分支
```

- 当 `sel == 0` 时，这个 `always` 块中不会对 `y` 赋值，于是 `y` 会保持之前的值。
- 这种「在某些条件下不赋值」的写法，在组合逻辑 `always` 中会被综合成一个锁存器，而不是一个纯粹的 `mux2`。
- 对于 2:1 组合 `mux`，应确保在所有输入组合下都给 `y` 赋一个确定值。

参考正确写法

```
1 module mux2(  
2     input    a,  
3     input    b,  
4     input    sel,  
5     output reg y  
6 );  
7     always @(*) begin  
8         if (sel)           // 等价于 if (sel == 1'b1)  
9             y = a;  
10        else  
11            y = b;  
12    end  
13 endmodule
```

或用连续赋值：

```
1 module mux2(  
2     input a,  
3     input b,  
4     input sel,  
5     output y  
6 );  
7     assign y = sel ? a : b;  
8 endmodule
```

考点总结：

- 组合逻辑用 `always @(*)`，敏感列表要包含所有输入。

- 数字常量建议写成 `1'b1`、`1'b0`。
- 组合逻辑里所有分支都要对输出赋值，否则会产生 latch。

练习 4 答案：模块实例化与 **testbench** 的边界

原 `adder2` 模块:

```
1 module adder2(  
2     input  [1:0] a, b,  
3     output [2:0] sum  
4 );  
5 // ... 省略实现  
6 endmodule
```

top 模块:

[illegible]

存在的问题

1. Q1: 模块实例化不能写在 `always` 里

```
1 | always @(posedge clk) begin
2 |     adder2 u1(...); // 错误
3 |     ...
4 | end
```

- Verilog 中，模块实例化是静态结构，在硬件中就对应一块固定电路。
- 不论时钟怎么跳变，硬件实例的数量都是固定的，不可能「每个周期创建一个 `adder2`」。
- 因此，实例化语句必须写在模块的结构层级上，而不是过程块 (`always`、`initial`) 里面。
- 正确方式：把实例化移到 `always` 外部。

2. Q2: `led` 声明为默认 `wire`，却在 `always` 里用过程赋值

```
1 | output [2:0] led; // 默认是 wire
2 | ...
3 | always @(posedge clk) begin
4 |     led = sum; // 对 led 做过程赋值
5 | end
```

- `output [2:0] led` 默认是 `wire` 类型。
- 在 `always` 块里做 `led = sum;` 是过程赋值，左边必须是 `reg`/integer/time 等「变量类型」，不能是 `wire`。
- 因此要么：
 - 把 `led` 声明成 `output reg [2:0] led;`，并在时钟沿上更新；
 - 要么不用在 `always` 里赋值，而是用 `assign led = sum;`（此时 `led` 仍为 `wire`）。

3. （顺带一提）`sum` 更适合作为 `wire`

```
1 reg [2:0] sum;
2 ...
3 adder2 u1(.a(sw_a), .b(sw_b), .sum(sum));
```

- `adder2` 的 `sum` 是组合逻辑输出，应该驱动一个 `wire`。
- 在 `top` 里把 `sum` 声明为 `wire [2:0] sum;` 更清晰，通常不会声明为 `reg`。
- 此处不是最致命的语法错误，但从设计规范角度来说不太好。

参考正确写法（典型同步输出）

```
1 module top(
2     input      clk,
3     input [1:0] sw_a,
4     input [1:0] sw_b,
5     output reg [2:0] led
6 );
7     wire [2:0] sum;
8
9     // 组合加法器的实例化放在 always 之外
10    adder2 u1(
11        .a    (sw_a),
12        .b    (sw_b),
13        .sum   (sum)
14    );
15
16    // 在时钟上升沿把结果寄存在 led 上
17    always @(posedge clk) begin
18        led <= sum;
19    end
20 endmodule
```

如果你根本不需要时钟寄存，只想「开关变，LED 立刻跟着变」，也可以写成纯组合：

```

1  module top(
2      input  [1:0] sw_a,
3      input  [1:0] sw_b,
4      output [2:0] led
5  );
6      adder2 u1(
7          .a    (sw_a),
8          .b    (sw_b),
9          .sum  (led)
10     );
11 endmodule

```

这里：

- `adder2` 在模块体内、所有 `always` 之外，静态实例化。
- `led` 没有过程赋值，保持 `output [2:0] led;`（`wire`）即可。

如果你愿意，下一步可以试试：

你自己把改正后的四个模块手敲一遍，再自己检查一遍「`wire/reg`」「`assign/always`」「组合/时序」「`latch` 条件」这些关键词。有什么不太踏实的地方，我们再一点点拆开讲。🌱

可复制的 Typora LaTeX+Markdown 版本

```

1  ## 练习 1
2
3  **错误:**
4
5  1. 模块名不能以数字开头, `module 2and` 非法; 并且模块头结尾缺少分号, 应写成:
6
7  ```verilog
8  module and2(
9      input  a,
10     input  b,
11     output y
12 );

```

1. 端口已经声明 `output y`，内部又写 `wire y;`，同名重复声明。
2. `assign y = a & b` 末尾缺少分号。

参考正确写法：

```
1 module and2(  
2     input  a,  
3     input  b,  
4     output y  
5 );  
6     assign y = a & b;  
7 endmodule
```

练习 2

错误：

1. `output reg y;` 后又写 `wire y;`，同名不同类型，错误。
2. 在 `always @(*)` 里写 `assign y = a & b;` :
 - `assign` 不能出现在 `always` 里。
 - `assign` 左边必须是 `wire`，这里的 `y` 被声明为 `reg`。
3. `y <= ~a & b;` :
 - 组合逻辑里推荐用阻塞赋值 `=` 而不是非阻塞 `<=`。
 - 功能上也没实现 `a & b | ~a & b`，只留下了 `~a & b`。

参考正确写法（组合）：


```
1 module good_comb(  
2     input a,  
3     input b,  
4     output y  
5 );  
6     assign y = (a & b) | (~a & b);  
7 endmodule
```

练习 3

错误：

1. `always @(sel)` 敏感列表不完整，y 还依赖 a、b，应写成 `always @(*)`。
2. `if (sel == 1)` 数字常量不规范，建议写成 `if (sel)` 或 `if (sel == 1'b1)`。
3. 缺少 else 分支，当 sel 为 0 时 y 不被赋值，会综合出 latch，而不是纯组合 mux。

参考正确写法：

```
1 module mux2(  
2     input a,  
3     input b,  
4     input sel,  
5     output reg y  
6 );  
7     always @(*) begin  
8         if (sel)  
9             y = a;  
10        else  
11            y = b;  
12        end  
13 endmodule
```

练习 4

错误：

1. `adder2 u1(...)` 被写在 `always @(posedge clk)` 里，模块实例化必须在模块体的结构层级，不能在 `always` 里。
2. `output [2:0] led` 默认是 `wire`，却在 `always` 里用过程赋值 `led = sum;`，左边应是 `reg` 类型，或者把赋值改成 `assign`。

（顺带：`sum` 更适合声明为 `wire [2:0] sum;`，由组合加法器输出驱动。）

参考正确写法（同步寄存结果）：

```
1  module top(  
2      input      clk,  
3      input  [1:0] sw_a,  
4      input  [1:0] sw_b,  
5      output reg [2:0] led  
6  );  
7      wire [2:0] sum;  
8  
9      adder2 u1(  
10         .a    (sw_a),  
11         .b    (sw_b),  
12         .sum  (sum)  
13     );  
14  
15     always @(posedge clk) begin  
16         led <= sum;  
17     end  
18 endmodule  
19
```